Session 2 - Modern LLM Internals & SFT: Foundation II

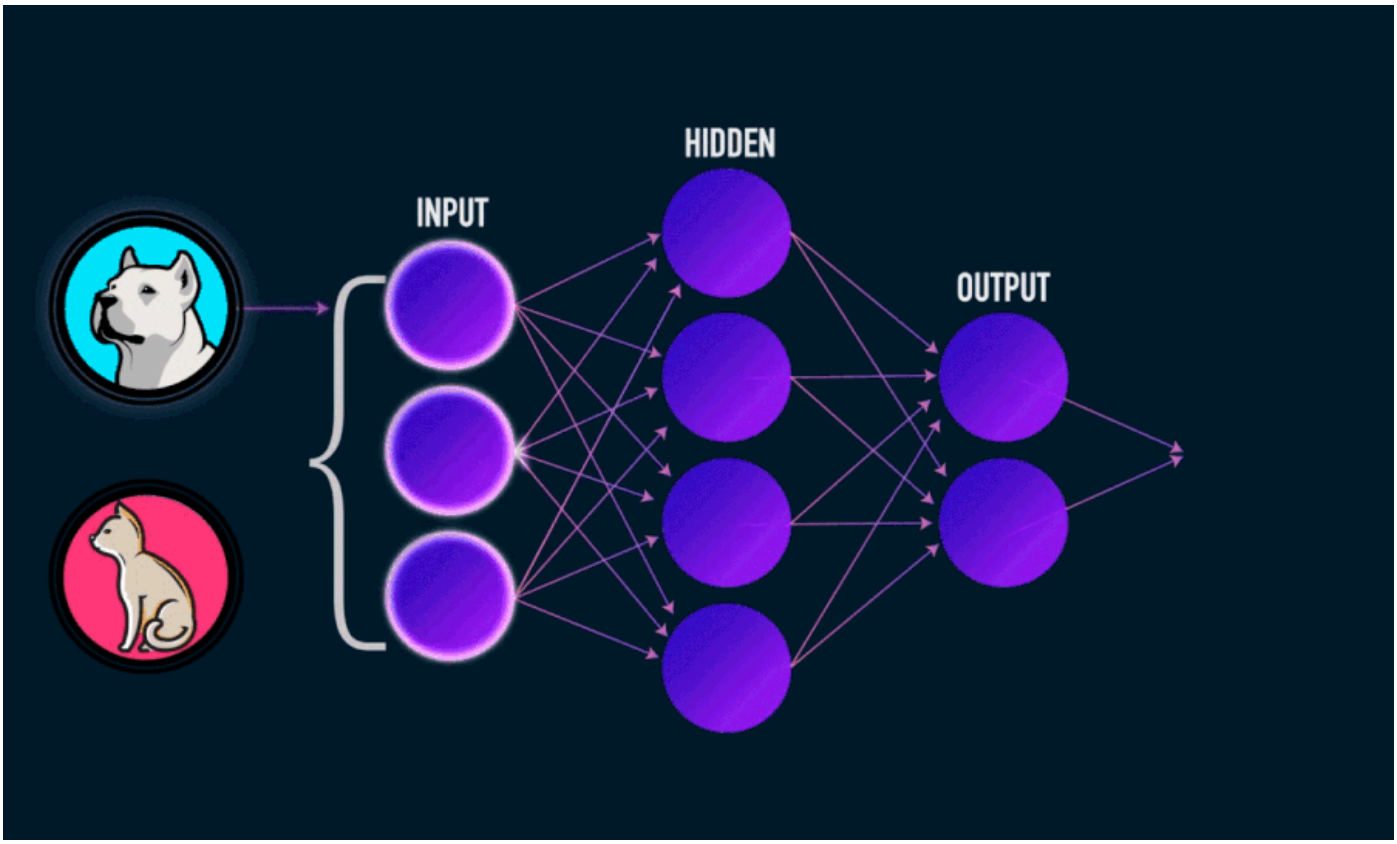
Session 2 - Modern LLMs & SFT

Foundation Part - II

Let's do a step-by-step **recap** of the last session.

We saw that solving complex/non-linear problems is easy for neural networks, and that they can solve any problem, as NNs are **Universal Approximators**.

We also discussed that we used **Back Propagation** to train NNs.



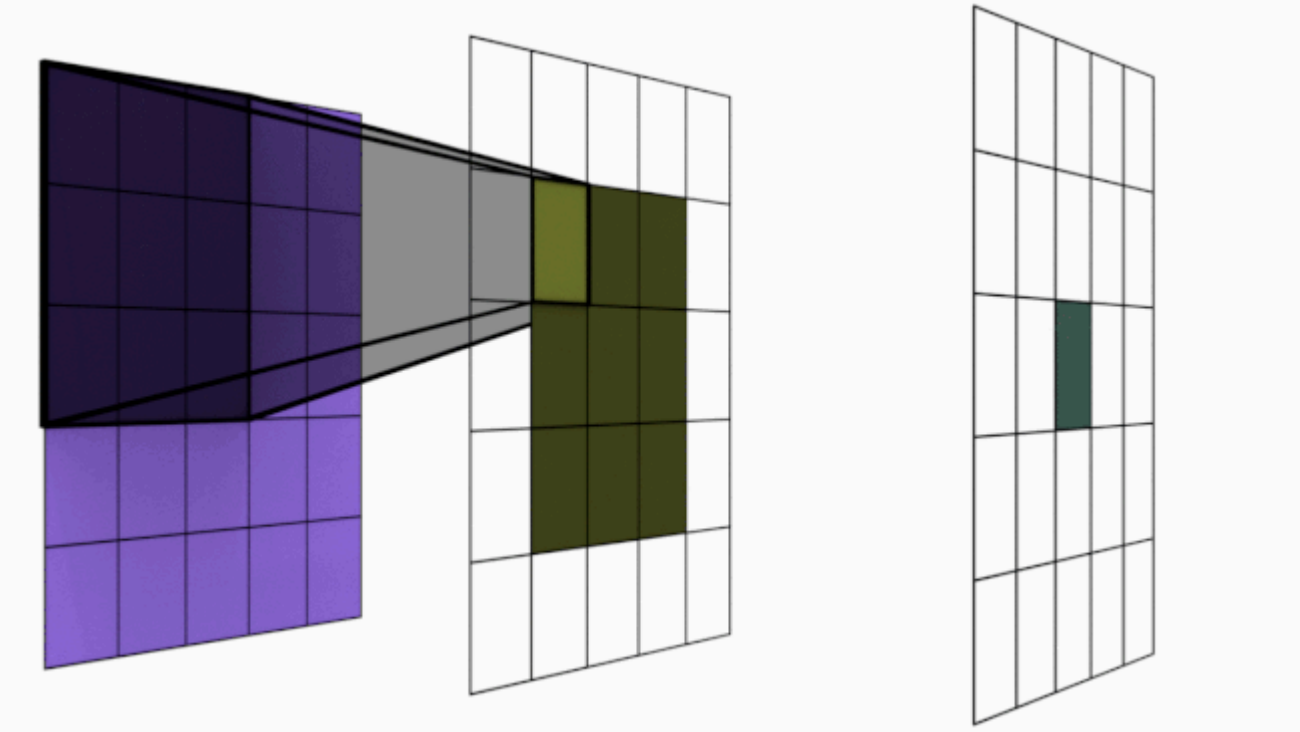
In BackPropagation we have this thing called "**learning rate**" that allows us to provide balanced feedback to weights to get "slightly" better next time.

By 2016 we were doing pretty **good** on **images**, but **lagging** a lot behind on **text**.

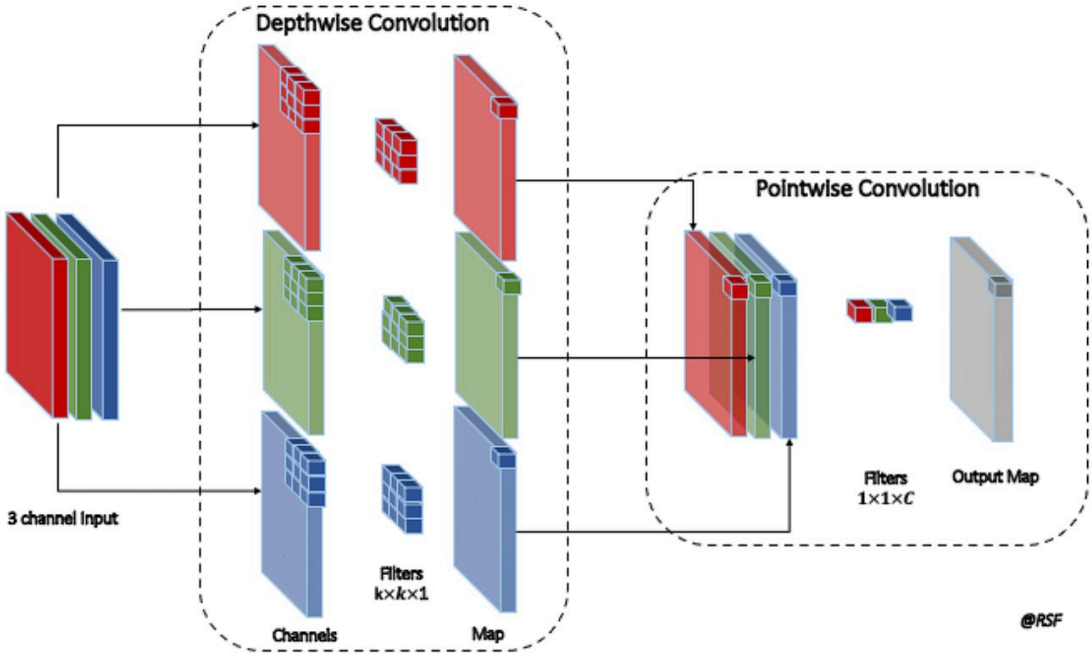
The problems we had on text were:

- for **longer** sentences, **nothing worked**
- for **each problem** (translation, summarisation, translation, spam detection, QnA, etc), we had **different models**
- making **models larger** didn't make models necessarily better
- we didn't have **large annotated datasets** for training LLMs

Before we proceed, let's look at what Convolutional Neural Networks were doing to make things better:

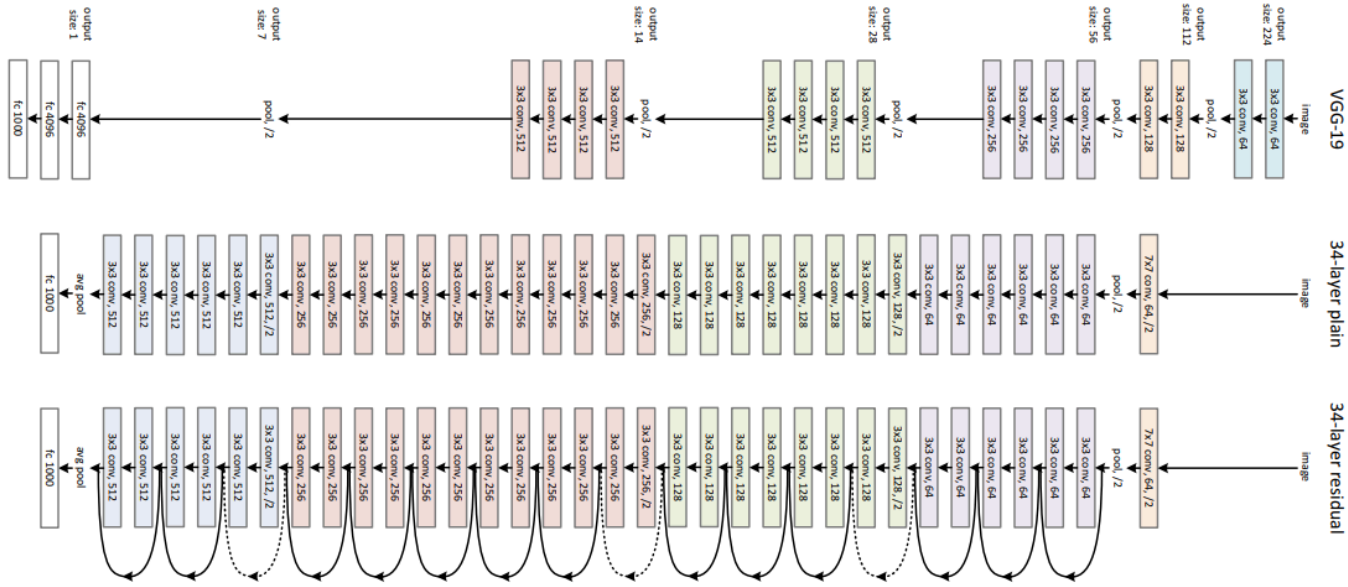


Above is the most important image for CNNs. As you can see, each of the next Yellow pixel is getting "**localized**" data from a fixed location. So throughout the network, weights "knew" the location of incoming data.



The input data had "channels" and it was processed in channels. Incoming images had 3 channels, but we would increase those to 2048 as well in large models.

But the biggest breakthrough in images we had was in 2015, when RESNET model was introduced:



In the last row you can observe that we have these lines with arrows, called "skip connections". These skip connections can short circuit anything in between and let the backpropagation sent feedback to any weight from the loss calculations! This allowed CNNs to go up to 1000 layers (we never had enough imaging data to go beyond and make models usable).

3 things that I want you to remember from above are:

1. Location Knowledge made CNNs work
2. Channels allowed to segregate concepts
3. Skip connections allowed for deeper models.

Now this transformer model below is obsolete nowadays. You're looking at an encoder and a "specific" version of decoder.

Today **modern decoders** look like this:

If you ever want to go deeper into the architecture and look at it, then there's no link better than **this** (https://bbycroft.net/llm) .

Let's discuss each text item in this image, but can you see "location, channels and skip connections"? :

- Input: a sequence of:
  - text, represented via words (we'll talk about tokens later)
  - image, represented via patches of images
  - audio, represented via patches of **audio** (https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgLBj7HiE4JZhOjRMZw2PdpCSalQnLEYRv_bY21rHlikUUjQAvs982fuXKJwznwLtwgmMYTSgE2M-NrGdB7X_ALZlEsUOfe63HuldppfbAziJm32foIMhGX-FEw2e-PnmmGH42OgPnyU2Y/s0/yamnet_animation+%25282%2529.gif) converted into **mel-spectogram** (https://ars.els-cdn.com/content/image/1-s2.0-S0045790624000284-gr3.jpg)
  - video, sequence of patches of video frames
  - time-series data, a sequence of time-stamped numerical values
  - **DNA** (https://engineering.berkeley.edu/news/2025/02/new-ai-breakthrough-can-model-and-design-genetic-code-across-all-domains-of-life/) , sequence of base pairs (GCTA)

  - code, sequence of words, and special symbols

- Embeddings: each of "one" item above needs to be represented into something that a model can understand:
  - it cannot be just a number as that doesn't have a lot of information
  - Problem here is, in how many ways can we decide the top row (also called embedding dimensions), and how do we find the right ones!
  - We solved it in 2003 (scroll up for word-embeddings). We allow the computer to determine what these dimensions stand for. And unlike us (who think in discrete values, like 1, 2, 3, love, hate), AI models can have extremely complex and compact representations (for example, think of 15 vs 1111, and embeddings can be 0.1, -0.56, 0.00012, -0.5423)
  - We now a days only tell our model "total dimensions", larger the better:

- Position Encoding: we "tag" our sequence inputs with their positional values:
  - this is required as when we send a lot of data, the model needs some way to identify what came before or later.

- Attention: the core algorithm that has made all of this development possible. This algorithm:
  - evolves the meaning of a word based on its surrounding work
    - ***Dhoni is nothing less than a tiger. And this tiger won India a world cup!*** The meaning of the word ***tiger*** is different in both the sentences.

      Also analyse this:

      I went to the bank, which was next to the bank of Ganges, and on the left was a banking road.

      Attention mechanism identifies the correct meaning of the word based on the text around it

  - finds which part of the sequence is important, and what it should focus on. In the above sentence, if we ask "Who was the Tiger?", LLMs today can tell us, that we were referring to Dhoni.
  - **has neural networks inside not shown**

- Multi-head: we saw above that our "dimensions" can run into 10000s! We don't want to deal with all of them at the same time. So we dissect them, say into pieces of 128, or 258, and then process them. Helps in Parallel Processing and "contextuality".

- Add: addition, like 1 + 4 = 5 😎 see, not everything is new here

- Norm: normalizes things:
  - We like everything to be between -1 and +1, and we are adding things.. so 0.9, -0.8, 0.4 and 0.5, -0.5, 0.8 can become 1.4, -1.3 and 1.2. We convert that to 1, -1 and 0.85.

- Feed Forward: remember we "dissected" our embeddings? Here we combine them to mix "context" and build the whole thing (can't process contexts in isolation)
- Nx: Repeat N times

- Linear: Final Neural network Layer, that predicts

- Softmax: Prediction can be like 5, 0.4, 1, 2.3, 4.5.. Softmax converts that into 0.58, 0.0059, 0.0108, 0.0395, 0.3563, which sums up to 1

- Output Probabilities: Above numbers only.

Moving On...

Let's talk about Tokens first

We need a way to feed out text to our LLMs. We have a few options:

characters,

words,

or tokens

"**Tokenization is an amazing piece of engineering!**"

This sentence has 48 characters, or

8 words or

9 tokens.

If we work with characters, we have only a **few** (https://en.wikipedia.org/wiki/ASCII#Table_of_codes) hundred inputs and a few **hundred** (https://en.wikipedia.org/wiki/UTF-8) **outputs** (https://www.fileformat.info/info/charset/UTF-8/list.htm). But our LLMs would need to run 47 times!

If we work with words, we have only 7 words, but then we need to predict 1 in the millions of words, but LLM would work 8 times.

But, if we work with tokens, then we need to predict 1 in 50k, or 100k, or 200k tokens, and LLM would work **9 times** (https://tiktokenizer.vercel.app/?model=o200k_base)!

Tokenization is at the core of LLMs. Tokenization is the process of finding the most common repeating characters that come together, which can allow us to make "any" text that exists in "any" language!

The reason there is no first "Hindi" LLM is because all tokenizers start with using English words, and then focus on the **remainder** languages.

Let's look at the list of **top 200 (https://time.com/collection/best-inventions-2024/)** products/innovations of 2024 and find India to get some context.

Let's play a little with tiktoken to see how it performs on this text:

Tokenization is at the heart of much weirdness of LLMs. Do not brush it off. Also, notice space is part of the token.

127 + 677 = 804
1275 + 6773 + 8041

Egg.
I have an Egg.
egg.
EGG.
Egg?

इस वाक्य का अनुवाद गूगल ट्रांसलेट द्वारा किया जा रहा है।

We can see here that the number of tokens for other languages is much higher.

```
for i in range(1, 101):
    if i % 3 == 0 and i % 5 == 0:
        print("Fizzbird")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

If you're interested to learn more about tokenization, then go through this **amazing video (https://www.youtube.com/watch?v=zduSFxRajkE)** by Andrej Karpathy!

What are LLMs or GenAI models in general?

All modern LLMs or GenAI models are basically doing this:

Predict the next token.

Again, important things here to remember are that for each of the next token prediction, "all of the previous tokens must be available". This will become crucial when we move to agentic AI concepts, discuss how we handle context lengths.

Millions of input tokens but only a few k-output tokens?

The difference between input tokens (2M) and output tokens (16k) in models like Gemini comes down to these facts:

- Processing vs Generation: A model can **read** and **process** a massive amount of tokens (let's see how)
- Memory & Computation Limits: Reading tokens is "free" but generating each token requires sampling and ranking multiple possible next tokens

In short, nowadays models are designed to **read** huge amounts of data, but optimized to **generate** only as much as is typically useful. The computational cost of decoding scales differently than encoding.

But why do we call them LLMs?

They have a **large** number of parameters running into billions

They are designed to understand, generate and manipulate **language**

At their core, they are machine learning **models**.

Pretraining Objectives of LLMs

Casual Language Modeling (CLM) - "Next Token Prediction

Source (https://arstechnica.com/ai/2024/12/why-ai-language-models-choke-on-too-much-text/)

Given a sequence of tokens (words, subwords, or characters), the model learns to predict the next token at each position. (Question to you, during training we use "masking", do we use masking while inference. Also, do you know the difference between Masking and inferencing?)

Pretraining objective is simple, and we now have infinite data!

But there is something way more interesting thing that you may want to know about!

laws!

Scaling Laws

Scaling laws are the reason why Microsoft gave billions to OpenAI, and everyone is spending so much.

The law says "if we train larger models over more data, we get better results".

But more than just text, there's an equation that we need to look at. But before you look at the equation look at this graph:

What you're seeing above is that we can "predict the final loss" for fixed compute and model parameters. But not only that, if you can predict for a smaller model, then you can predict for a larger model with the same architecture as well! This is insane!

This means that you CAN train a small model at low cost, while performing all your experiments, and then proceed to train a larger model and get predictable performance!

Chinchilla Scaling Law:

Human Language: $Loss = 1.69 + \frac{406.4}{ModelSize} + \frac{410.7}{Compute}$

PhD Language: $L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$,

Basically, we can calculate this table:

And to keep things simple:

- Number of Training Tokens = 20 * total_paremeter_count
  - 400M Model → 8B tokens
  - 1B Model → 20B tokens
- Relationship between parameters in our model vs FLOPs
  - total_palm_flops = (6 * total_parameter_count + 12 * num_hidden_layers * num_attention_heads + head_dim * tokens_per_sample)* tokens_per_sample

Unfortunately, law also predicts how badly other models were trained:

Why Fine Tuning?

But what you get after CLM is something that predicts the next token.

Sentence Completion -

Prompt: The cat sat on the
PreTrained: window, watching the birds outside as the sun began to set.
FineTuned: mat, purring softly as the sun warmed its fur.

Basic Match Question -

Prompt: What is 2 + 2?
PreTrained: a common arithmetic problem often taught to young students.
FineTuned:4.

Factual Question -

Prompt: Who is the president of the United States?
PreTrained: one of the most powerful political figures in the world, elected every four years in the United States.
FineTuned: As of 2025, the president of the United States is [*correct answer* 😅].

Joke Setup -

Prompt: Why did the chicken cross the road?
PreTrained: Chickens are known to move across roads due to their natural behaviour of foraging.
FineTuned: To get to the other side!

Instruction Following -

Prompt: Write a professional email requesting a meeting.
PreTrained: Then check with the recipient whether they have received the email of not.
FineTuned: Dear [Recipient], I hope you're doing well. I'd like to schedule a meeting to discuss [topic]. Please let me know your availability. Best regards, [Your Name].

Code Generation -

Prompt: Write a Python function to add two numbers.
PreTrained: Python is a widely used programming language known for its readability and simplicity
FineTuned:

```python
def add(a, b):
    return a + b
```

As you can see above, a PreTrained model is not necessarily very useful. It has no charm or fun, it's boring, it's competing with what you're about to say, it never understands a joke, nor does it understand the gravity of things. Don't believe me, here actual proof:

Supervised Fine Tuning!

What SFT Does?

During SFT, the model is fine-tuned using a high-quality dataset made up of examples like:

Input Instruction	Ideal Response
"Summarize this article for a 5th grader"	Clear, age-appropriate summary
"Write Python code to sort a list"	Correct, efficient code with explanations.
"Explain photosynthesis to a beginner"	Consize, scientifically accurate explanation

How do we do SFT?

Well, now we change the dataset from random internet articles to "instruction following examples".

Notable Real Datasets Used in SFT

Dataset	Description	Example Contribution
OpenAssistant Conversations (https://huggingface.co/datasets/OpenAssistant/oasst1) (2023)	161k crowd-sourced dialogues designed to train instruction-following LLMs	Real-world, diverse humar queries.
Dolly 2.0 (https://www.databricks.com/blog/2023/03/24/hello-dolly-democratizing-magic-chatgpt-open-models.html) (Databricks, 2023)	Open dataset of 15k high-quality human-created instruction-response pairs	Focuses on business, coding, reasoning tasks
Anthropic's HH-RLHF Pre-SFT Data (https://huggingface.co/datasets/Anthropic/hh-rlhf) (2023)	~100k dialogue examples annotated for helpfulness, honesty, harmlessness	Direct focus o aligning assistant-style behavior
FLAN Collection (https://research.google/blog/the-flan-collection-advancing-open-source-methods-for-instruction-tuning/) (Google, 2022-23)	Finetuning data across hundreds of task types (summarization, reasoning, translation, etc.)	Expands the model's breadth acros task formats
InstructGPT (https://openai.com/index/instruction-following/) Dataset (OpenAI, 2022)	Proprietary data used to finetune GPT-3 into InstructGPT	Directly teaches instruction-following with

		human demonstration

None of the instructions following the datasets are made in India, following instructions is not something we like to do. So no, sorry, won't make one. Never!

How Much Compute Does Post-Training Need?

- Pre-training: 98% of total compute (hundreds of GPU years for models like GPT-4 and LLaMa 3)
- Post-training (SFT + RLHF): ~2% of total compute
  This makes SFT an incredibly **efficient way to unlock the capabilities that are already hidden inside the pre-trained model**.

Assignment

1. You're making one more Chrome Plugin, but this time, with Google Gemini Flash Lite 3.0 integrated! (Let's try some experiments first)
2. Think of some amazing idea, and then make a plugin where you use this LLM
3. Show it working in a YouTube video and share the link on the LMS. [250pts]
4. If you share it with the world (LinkedIn/X/Medium), then 250 additional points (optional)

Video

Studio (**Transcript (https://canvas.instructure.com/courses/14531601/files/344766045?wrap=1)** **(https://canvas.instructure.com/courses/14531601/files/344766045/download?download_frd=1)**)

GMeet